PROJECT SAFECRACKER: DIGITAL COMBINATION LOCK USING ESP32-C6 AND A ROTARY ENCODER

A PROJECT BASED LEARNING REPORT

Submitted by

Naresh A [210425105035]

Vishnu Varthan V G [210425105062]

This project report is submitted as part of the requirements for the completion of

EE5304 - Microprocessor and Microcontroller



DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

CHENNAI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SARATHY NAGAR, KUNDRATHUR – 600 069

TAMIL NADU, INDIA

OCTOBER 2026

Vision of the Institution

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institution

IM1: To create next generation leaders by effective teaching learning methodologies and instil scientific spark in them to meet the global challenges.

IM2: To transform lives through deployment of emerging technology, novelty and sustainability.

IM3: To inculcate human values and ethical principles to cater to the societal needs.

IM4: To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.

IM5: To nurture incubation centres enabling structured entrepreneurship and start-ups.

Vision of the Department

To Excel in the emerging areas of Electrical and Electronics Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to meet the industrial and societal needs

Mission of the Department

DM1: To provide strong fundamentals and technical skills through effective teaching learning Methodologies.

DM2: To transform lives of the students by fostering ethical values, creativity and novelty to become entrepreneurs and establish start-ups.

DM3: To habituate the students to focus on sustainable resources with optimal usage to ensure the welfare of the society.

DM4: To provide an ambience for research through collaborations with industry and academia.

DM5: To inculcate learning of emerging technologies for pursuing higher studies leading to lifelong learning.

CHENNAI INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi & Affiliated to Anna University Chennai), Sarathy Nagar, Kundrathur, Chennai - 600069.

BONAFIDE CERTIFICATE

Certified that this project report entitled **"PROJECT SAFECRACKER: DIGITAL COMBINATION LOCK USING ESP32-C6 AND A ROTARY ENCODER"** is the Bonafide work of **Naresh A [210425105035], Vishnu Varthan V G [210425105062]** who carried out the EE5303 Control System Engineering - Project Based Learning under my supervision during the academic year **2026-2027**.

SIGNATURE

Dr. N. MALATHY, M.E. Ph.D.,

HEAD OF THE DEPARTMENT

Assistant Professor,

Department of Electrical and Electronics Engineering,

Chennai Institute of Technology,

Chennai – 600069.

SIGNATURE

Ms. M. KANIMOZHI, M.E,

SUPERVISOR

Assistant Professor,

Department of Electrical and Electronics Engineering,

Chennai Institute of Technology,

Chennai-600069.

Submitted for Project Viva Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is our first and foremost duty to express our heartily gratitude and allegiance to our beloved chairman **Sri. P. SRIRAM** for providing us a good working environment, facilities and opportunity to carry out the research work with full freedom in and out of the campus.

We would like to express our deepest gratitude to our Principal **Dr. A. RAMESH, Ph.D.,** for being our inspiration throughout the entire period of course.

We greatly appreciate and convey our heartfelt thanks to **Dr. N. MALATHY, M.E, Ph.D.,** Assistant Professor & Head – Electrical and Electronics Engineering, for her continuous support and valuable suggestions throughout the duration of our core course project.

We express our gratefulness to **Dr. J. POORNIMA, M.E, Ph.D.,** Assistant Professor and Program Coordinator, Department of Electrical and Electronics Engineering, Chennai Institute of Technology for her constructive criticism in the reviews.

It gives us an immense pleasure to express our deep sense of gratitude to our project guide, **Ms. M. KANIMOZHI, M.E,** Assistant Professor, Faculty of Electrical and Electronics Engineering for her valuable guidance, constant inspiration, encouragement, and motivation.

We express our gratitude and sincere thanks to all the faculty and supporting staff of Department of Electrical and Electronics Engineering for their valuable suggestions and support during the course of our project work.

Naresh A [210425105035]

Vishnu Varthan V G [210425105062]

ABSTRACT

Combination locks protect safes, lockers and doors, and their electronic versions need a reliable way of reading the operator's input and a clear logic that decides when to open. This project presents Project Safecracker, a digital combination lock built on the Espressif ESP32-C6 microcontroller. A quadrature rotary encoder with a push button acts as the dial. Its two phase-shifted channels, CLK and DT, are decoded in interrupt service routines that combine the previous and the present states into a 4-bit number, so that the counter changes only for the eight legal clockwise and counter-clockwise transitions and contact bounce is filtered without any blocking delay. The counter wraps between 0 and 15, and the operator confirms each digit with the push button. A finite state machine with three modes, LOCKED, SUCCESS and ALARM, compares the three-digit entry with a stored passcode, counts the failed attempts and moves to a lockdown after the third failure. A 0.96-inch SPI OLED display shows the secure-terminal interface with the digit being dialled and the masked digits already entered. The condition of the lock is shown by two ordinary LEDs, a red one for the locked and alarm conditions and a green one for the unlocked condition, and an active buzzer gives a click for every step, a confirmation tone for every digit and a pulsing alarm. The report presents the circuit, the program, the working principle, the decoding table of the encoder and the calculation of the combination space of 4096 codes, together with the practical points that make the circuit work. The system is inexpensive and easy to build, and it can be extended with non-volatile storage of the passcode, a timed lockout, a real bolt and wireless alerts.

Keywords: Combination lock, ESP32-C6, rotary encoder, quadrature decoding, interrupt service routine, finite state machine, SSD1306 OLED, LED indicator

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
ABSTRACT	**vi**	
LIST OF FIGURES	**viii**	
LIST OF TABLES	**ix**	
LIST OF ABBREVIATIONS	**x**	
1	**INTRODUCTION**	**1**
1.1	BACKGROUND	1
1.2	PROBLEM STATEMENT	3
1.3	OBJECTIVES	3
1.4	SCOPE OF THE PROJECT	3
1.5	PROPOSED SOLUTION	4
2	**LITERATURE SURVEY**	**5**
2.1	EXISTING SYSTEMS	5
2.2	LIMITATIONS OF EXISTING SYSTEMS	6
2.3	COMPARISON OF EXISTING APPROACHES	6
2.4	RESEARCH GAP	6
3	**SYSTEM REQUIREMENTS**	**7**
3.1	HARDWARE REQUIREMENTS	7
3.2	SOFTWARE REQUIREMENTS	8
4	**SYSTEM DESIGN**	**10**
4.1	BLOCK DIAGRAM	10
4.2	CIRCUIT DIAGRAM	11
4.3	STATE MACHINE AND FLOWCHART	13
5	**IMPLEMENTATION**	**15**
5.1	HARDWARE IMPLEMENTATION	15
5.2	SOFTWARE IMPLEMENTATION	16

6	**WORKING OF THE SYSTEM**	**21**
7	**RESULTS AND DISCUSSION**	**24**
7.1	EXPERIMENTAL SETUP	24
7.2	OBSERVED RESULTS	24
7.3	DISCUSSION	26
8	**ADVANTAGES, LIMITATIONS AND APPLICATIONS**	**27**
8.1	ADVANTAGES	27
8.2	LIMITATIONS	27
8.3	APPLICATIONS	27
9	**CONCLUSION AND FUTURE SCOPE**	**28**
9.1	CONCLUSION	28
9.2	FUTURE SCOPE	28
REFERENCES	**29**	
OUTCOMES OF PROJECT	**30**	

LIST OF FIGURES

FIGURE NO. AND TITLE **PAGE NO.**

1.1 Quadrature signals of the rotary encoder for both directions of rotation 1

4.1 Block diagram of the proposed system 10

4.2 Circuit diagram of the proposed system 11

4.3 State diagram of the security state machine 13

4.4 Flowchart of the system operation 14

6.1 Illustrative rejection of contact bounce by the state-tracking decoder 22

7.1 Chance of opening the lock by guessing within three attempts 25

LIST OF TABLES

TABLE NO. AND TITLE **PAGE NO.**

2.1 Comparison of existing approaches 6

3.1 Hardware requirements 7

3.2 Software requirements 8

4.1 Pin connections of the ESP32-C6 with the modules 12

4.2 States of the security state machine 13

5.1 Common problems and remedies 20

6.1 Decoding of movement State (previous phase and present phase) 21

6.2 Outputs of the lock in each state 23

6.3 Entry of the correct passcode {9, 1, 15} 23

7.1 Expected behaviour of the lock at each stage 24

7.2 Functional tests of the lock 24

7.3 Combination space and chance of guessing for a 16-position dial 25

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CLK	Clock (encoder channel A)
CS	Chip Select
DC	Data / Command
DT	Data (encoder channel B)
FSM	Finite State Machine
GND	Ground
GPIO	General-Purpose Input / Output
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IRAM	Internal RAM
ISR	Interrupt Service Routine
LED	Light-Emitting Diode
MCU	Microcontroller Unit
MOSI	Master Out, Slave In
OLED	Organic Light-Emitting Diode
PBL	Project-Based Learning
RES	Reset
RISC-V	Reduced Instruction Set Computer, fifth generation
SCK	Serial Clock
SoC	System on Chip
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SW	Switch (encoder push button)
USB	Universal Serial Bus
VCC	Positive Supply Voltage

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Locks and access-control devices are among the oldest security mechanisms used by people. The traditional combination lock of a bank vault or a school locker relies on a rotating dial: the operator turns the dial to a series of positions in a fixed order, and only the correct sequence releases the bolt. Modern safes, hotel strong-boxes and door controllers have replaced the purely mechanical parts with electronics, but the idea has not changed. A secret combination is entered through some form of physical input, compared with a stored code, and the result decides whether the system opens or stays locked.

A digital combination lock therefore combines four different engineering topics in one small system: a sensor that follows the motion of the operator's hand, a way of cleaning and interpreting the noisy signals of that sensor, a decision logic that remembers what has happened so far, and an output stage that tells the operator what the system is doing. This project, called **Project Safecracker**, builds such a lock on a modern 32-bit microcontroller and uses it as a vehicle for studying each of those topics in a practical way.

1. Rotary Encoders and Quadrature Signals

A rotary encoder is a sensor that converts the angular movement of a shaft into electrical pulses. The type used here is the incremental quadrature encoder. It has two output channels, called CLK (channel A) and DT (channel B), and a built-in push button, SW. When the shaft is turned, the two channels produce square waves of the same frequency that are shifted in phase by 90°. The pair of logic levels therefore passes through the four states 00, 10, 11 and 01 in one order for one direction of rotation, and in the reverse order for the other direction. Such a sequence, in which only one bit changes at a time, is a form of Gray code [9]. Figure 1.1 shows the two channels for both directions of rotation. The channel that changes first tells the controller which way the shaft is turning, and the number of changes tells it how far.

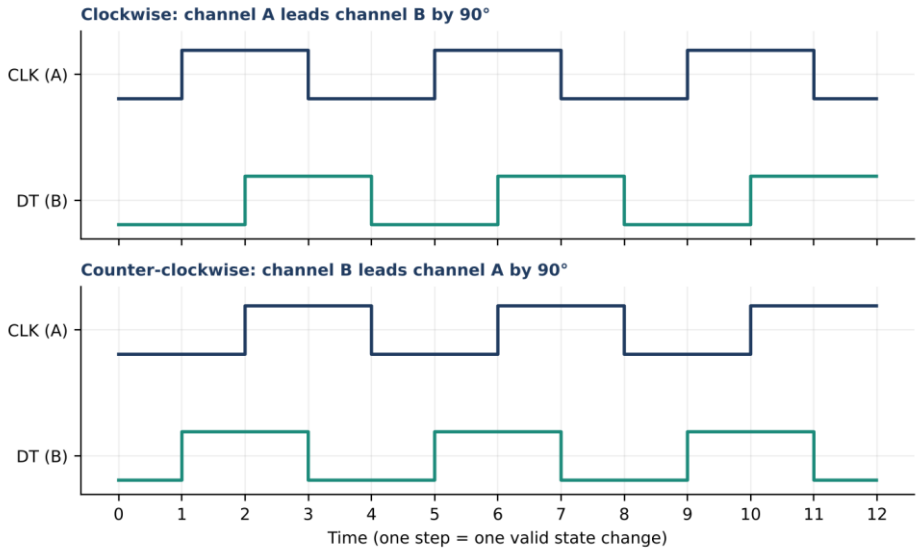


Figure 1.1 Quadrature signals of the rotary encoder for both directions of rotation

2. Contact Bounce and Its Remedy

The contacts inside a low-cost mechanical encoder do not close cleanly. When a contact makes or breaks, it vibrates for a short time and the signal flickers between high and low before it settles. This effect is called contact bounce [6], [7]. If the controller counts every edge it sees, one small movement of the knob can be registered as several steps, in the correct or even in the wrong direction. A common remedy is to wait for a fixed delay after each edge, but the delay stops the processor from doing anything else and the knob can feel sluggish. A better remedy, used in this project, is to remember the previous state of the two channels and to accept a new state only when the pair of old and new states forms one of the eight legal transitions of the quadrature sequence. A transition that is not legal is simply ignored. No waiting is needed, and the work is done inside an interrupt service routine so that no knob movement is missed.

3. Finite State Machines in Security Logic

A lock must behave differently depending on its history. Before the correct code is entered it must refuse access, after the correct code it must open, and after too many wrong attempts it must stop accepting input and raise an alarm. A finite state machine (FSM) describes this behaviour with a limited number of states and clearly defined rules for moving from one state to another [8]. The machine of this project has three states, LOCKED, SUCCESS and ALARM, and the transitions between them depend on the three-digit entry and on a counter of failed attempts.

4. Role of the Microcontroller and the Feedback Devices

The Espressif ESP32-C6 is a low-cost 32-bit RISC-V system-on-chip that provides many general-purpose input/output (GPIO) pins, hardware SPI and I2C interfaces, edge-triggered interrupts on every GPIO, and a 3.3 V logic level [1], [2]. In this project it reads the encoder, executes the state machine and drives the output devices. The operator receives

feedback in three ways. A 0.96-inch SPI OLED display shows the secure-terminal screen with the three digit slots and the current dial value. Two ordinary light-emitting diodes (LEDs) show the lock status at a glance: the **red LED** is on while the system is locked and the **green LED** is on when it is unlocked. An active buzzer gives an audible click for every step of the dial, a confirmation tone for every entered digit, and a continuous alarm in the lockdown mode.

5. Why Dial-Based Entry?

A keypad entry system lets the user type any digit at any time, while a dial forces the user to move through the values one after the other. This makes the interaction with the system more physical and also makes the design of the sensing and debouncing logic more interesting. Many everyday products use the same principle: bank vault dials, digital safes, industrial selector switches and the control knobs of car infotainment systems. For a project-based learning exercise the dial lock is attractive because the complete signal path, from the movement of the hand to the lighting of an LED, can be observed and measured stage by stage.

1.2 PROBLEM STATEMENT

Low-cost electronic locks and student prototypes of such locks suffer from the following recurring problems:

- **Unreliable input reading:** Encoders read by simple polling or by counting every edge give false steps because of contact bounce, so the digit shown to the user is not the digit that was dialled.
- **Blocking and sluggish programs:** The use of long delay() calls to hide the bounce makes the program miss movements of the knob and makes the display and the indicators react slowly.
- **Unclear system status:** A lock that does not clearly show whether it is locked, unlocked or in an alarm condition is confusing and, in a real application, unsafe.
- **No protection against guessing:** A lock that accepts unlimited attempts can be opened by trial and error. A failure counter and a lockout are essential.

A compact, well-documented solution is therefore required that reads a rotary encoder reliably, runs a clear three-state security logic, and gives immediate visual and audible feedback with simple, inexpensive parts. If these requirements are not met, the lock may open for a wrong code, stay closed for the right one, or give the operator no clue about what is happening. For learners an undocumented circuit is another obstacle, so the project also records the wiring and the practical points in detail.

1.3 OBJECTIVES

The Project Safecracker system sets the following engineering objectives:

- **Objective 1: Reliable Dial Input.** To decode the phase-shifted CLK and DT signals of a rotary encoder inside interrupt service routines and to use bitwise state tracking so that contact bounce does not change the count.
- **Objective 2: Three-State Security Logic.** To implement a finite state machine with the modes LOCKED, SUCCESS and ALARM, to compare a three-digit entry with a stored passcode, and to move to the alarm mode after three failed attempts.
- **Objective 3: Status Indication with Two LEDs.** To show the state of the lock with a red LED (locked or alarm) and a green LED (unlocked), supported by an active buzzer for audible feedback.
- **Objective 4: User Interface on an OLED.** To display the secure-terminal screen, the digit being dialled, the masked digits already entered and the lock status on an SPI OLED display.
- **Objective 5: Verification and Documentation.** To stream the state, the encoder value, the slot number and the failure count to the serial monitor and to document the circuit, the program and the observations so that the project can be repeated.

1.4 SCOPE OF THE PROJECT

The project covers the design, construction and testing of a prototype combination lock that is operated with a rotary encoder. The scope includes the interrupt-based decoding of the encoder, the entry of a three-digit combination in which each digit lies between 0 and 15, the comparison with a hard-coded passcode, the three-state machine, the control of the two LEDs and the buzzer, the display of the interface on the OLED, and the serial monitoring of the internal variables.

The prototype is built on a solderless breadboard and powered from a USB port for laboratory demonstration. The system indicates the locked and unlocked conditions with LEDs only; it does not drive a real bolt, solenoid or servo. The passcode is stored in the program and is not changed at run time.

Inclusions and Exclusions

- **Included:** quadrature decoding by interrupts, bitwise debouncing, 16-position dial, three-digit passcode, three-state FSM, failure counter, red and green status LEDs, active buzzer, SPI OLED interface, serial telemetry, and calculation of the combination space.
- **Excluded:** mechanical locking hardware, wireless access, encrypted or non-volatile storage of the passcode, a printed circuit board, and a tamper-proof enclosure.

1.5 PROPOSED SOLUTION

The proposed system uses the ESP32-C6-DevKitC-1 board as the control unit, a quadrature rotary encoder with a push button as the input device, a 0.96-inch SSD1306

OLED as the display, an active buzzer for sound, and two LEDs with series resistors as the status indicators. The solution works in four stages:

- **Input acquisition:** The CLK and DT pins of the encoder generate interrupts on every change. The interrupt service routine forms a 4-bit value from the previous and the present states of the two channels, and the counter changes by one only when this value belongs to a legal clockwise or counter-clockwise sequence. The counter wraps around between 0 and 15.
- **Digit entry:** The counter value is shown on the OLED as the current digit. Pressing the push button stores the digit, masks it with an asterisk on the display, moves to the next slot and resets the dial to 0.
- **Verification:** After the third digit the program compares the three stored digits with the passcode. A correct code moves the machine to SUCCESS. A wrong code increments the failure counter, and the third failure moves the machine to ALARM.
- **Indication:** In LOCKED the red LED is on, in SUCCESS the green LED is on and the red LED is off, and in ALARM the red LED blinks together with the buzzer while the OLED shows LOCKDOWN.

Organisation of the Report

Chapter 2 reviews the existing work and techniques related to encoder-based locks. Chapter 3 lists the hardware and software requirements, and Chapter 4 presents the system design with the block diagram, circuit diagram, state diagram and flowchart. Chapter 5 describes the hardware and software implementation, and Chapter 6 explains the working of the system step by step. Chapter 7 discusses the results, Chapter 8 lists the advantages, limitations and applications, and Chapter 9 gives the conclusion and the future scope.

CHAPTER 2
LITERATURE SURVEY

2.1 EXISTING SYSTEMS

The techniques used in this project come from several well-established areas: the coding of angular position, the debouncing of mechanical switches, the decoding of quadrature encoders, the design of finite state machines, and the use of small OLED displays with microcontrollers. This section reviews the sources and the typical existing systems that are closely related to the present work.

The coding principle of the encoder goes back to the reflected binary code patented by Gray [9], in which consecutive values differ in a single bit. This property is what makes a quadrature encoder safe to read: since only one channel changes at any instant, a controller never sees an ambiguous in-between value. Textbooks on digital design [8] present the finite state machine as the standard tool for sequential control, and treat the design of the state diagram, the state table and the output logic in a systematic way.

The problem of contact bounce is described in the practical literature on embedded design. Ganssle [6] surveys the behaviour of real switches and compares hardware and software remedies, while Horowitz and Hill [7] discuss the same problem from the viewpoint of circuit design and show how resistor–capacitor filters and latch circuits can be used. Both emphasise that the cheapest and most flexible solution is a software one, provided that the program does not waste processor time while it waits.

For the decoding of encoders on small microcontrollers, two widely used open-source libraries exist. The Encoder library of Stoffregen [10] uses a state table and works best when the two channels are connected to interrupt-capable pins. The rotary encoder library of Buxton [11] also uses a state table in which only legal sequences of states change the position, so that the bounce of a worn contact does not produce false counts. The routine of the present project follows the same idea in a compact form: the previous and the present states are combined in a 4-bit number, and only eight of the sixteen possible values change the counter.

On the platform side, the manufacturer's documents [1], [2] describe the ESP32-C6 and its development board, including the 3.3 V logic and the need to place interrupt code in internal RAM (the IRAM_ATTR attribute). The SSD1306 controller [3] drives the 0.96-inch OLED, the U8g2 library [4] provides the drawing functions, and the interrupt model is described in the Arduino reference [5].

Typical Existing Locks

- **Mechanical dial lock:** The combination is stored in the mechanical position of discs. It needs no power, but the combination cannot be changed easily, there is

no limit on the number of attempts and there is no indication of the state other than the position of the bolt.

- **Keypad electronic safe:** The code is typed on a keypad and checked by a microcontroller. The system usually blocks the input after some wrong attempts and sounds a beeper. The interaction is simple, but the keypad hides how the signal is processed.
- **Encoder-based hobby lock with polling:** The encoder is read in the main loop and a delay is used to hide the bounce. The program is simple, but it is slow and loses steps when the loop is busy with the display.
- **Library-based interrupt decoding:** The encoder is decoded with a ready-made library. The result is reliable, but the learner does not see how the state tracking works.

2.2 LIMITATIONS OF EXISTING SYSTEMS

The mechanical dial lock and the keypad safe are mature products, but they are closed designs that cannot be inspected or modified by a student. The encoder-based hobby locks that use blocking delays suffer from missed steps and from a slow display. The library-based designs hide the decoding algorithm, so the connection between the physical sequence of the encoder states and the counter is not visible. In addition, many simple projects give little feedback: the operator does not know whether the dial has moved, whether the digit has been accepted or how many attempts remain.

2.3 COMPARISON OF EXISTING APPROACHES

Table 2.1 compares the typical approaches described above with the proposed system.

Table 2.1 Comparison of existing approaches

Approach	Input reading	Bounce handling	Feedback and security
Mechanical dial lock	Mechanical discs	Not applicable	None; unlimited attempts
Keypad electronic safe	Matrix keypad	Software delay	Beeper; lockout after wrong codes
Polling encoder lock	Polling in main loop	Blocking delay()	Basic display; often no attempt limit
Library-based encoder lock [10], [11]	State-table library	Valid-sequence table	Depends on the application
Proposed system	Interrupts on CLK and DT	4-bit valid-sequence check	OLED, red/green LEDs, buzzer; 3-attempt lockout

2.4 RESEARCH GAP

The review shows that the individual techniques, namely Gray-coded encoders, state-table decoding, finite state machines and OLED interfaces, are well known. What is still useful, especially for teaching, is a compact and clearly documented implementation that brings them together in one working lock and explains every stage of the path from the movement of the knob to the indication on the LEDs. The project fills this gap with a minimum-component design around the ESP32-C6. It documents the decoding table, the state machine, the wiring and the practical points, and it uses only two ordinary LEDs as status indicators, which keeps the circuit simple and makes the state of the lock obvious from a distance.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

This section lists the physical components required to build the lock. Table 3.1 gives each component with its model or rating and its function, and the main components are described in more detail below.

Table 3.1 Hardware requirements

S.No	Component	Model / Rating	Function
1	Microcontroller board	Espressif ESP32-C6-DevKitC-1	Reads the encoder, runs the state machine and drives the display, LEDs and buzzer
2	Rotary encoder	Quadrature encoder with push button (CLK, DT, SW)	Dial input and digit confirmation
3	OLED display	0.96-inch SSD1306, 128 × 64, SPI (7-pin)	Shows the secure-terminal interface
4	Status LEDs	5 mm red LED and 5 mm green LED	Red: locked or alarm. Green: unlocked
5	Resistors	2 × 220 Ω, 0.25 W	Limit the current of each LED
6	Active buzzer	3.3 V active buzzer module	Clicks, confirmation tones and alarm
7	Breadboard	Standard solderless board	Mounting and wiring of the prototype
8	Jumper wires	Male-to-male and male-to-female	Interconnection of the modules
9	USB cable	USB Type-C to Type-A	Power, programming and serial monitor

1. ESP32-C6-DevKitC-1

The ESP32-C6 is a 32-bit RISC-V system-on-chip made by Espressif that operates at a logic level of 3.3 V [1]. The DevKitC-1 board [2] carries the chip together with a USB connector, a 3.3 V regulator and the pin headers. All GPIO pins can be configured as interrupt sources, which is exactly what the encoder decoding needs. The board is powered from the USB port; the 5 V rail of the connector is available on a header pin and the regulated 3.3 V on another. Because the GPIO pins work at 3.3 V, all modules that are connected to the signal pins must also use 3.3 V logic.

2. Rotary Encoder

The encoder is a three-contact mechanical device with two quadrature outputs, CLK and DT, and a push-button output, SW. The module has pins for CLK, DT, SW, VCC and GND. In this project the pins are configured with the internal pull-up resistors of the ESP32-C6, so no external resistors are needed: each pin reads high when its contact is open and low when it is closed to ground. The push button is used to confirm a digit.

3. SSD1306 OLED Display

The 0.96-inch OLED module has 128 × 64 pixels and is based on the SSD1306 controller [3]. The version used in this project has seven pins (GND, VCC, SCK, SDA/MOSI, RES, DC and CS) and is connected through the hardware SPI bus. Each pixel emits its own light, so the module needs no backlight and the contrast is high. The display is driven with the U8g2 library [4] in full-buffer mode. Some modules have only four pins, because they use the I2C bus instead of SPI. The wiring and the code changes for that variant are given in Section 5.2.

4. Status LEDs and Resistors

Two ordinary 5 mm LEDs act as the status indicators of the lock. The red LED shows the locked condition and the alarm, and the green LED shows the unlocked condition. Each LED is connected between a GPIO pin and ground through a 220 Ω resistor. The resistor limits the current to a safe value, as calculated in Section 4.1. The longer lead of the LED, the anode, is connected to the resistor and the shorter lead, the cathode, to ground.

5. Active Buzzer

An active buzzer has its own oscillator and sounds at a fixed pitch whenever a voltage is applied to it. It is therefore driven directly by a digital output: HIGH switches the sound on and LOW switches it off. This makes it very simple to produce clicks of a few milliseconds and a longer tone for errors. Three-pin modules need the supply and ground pins to be connected as well as the signal pin; two-pin buzzers are connected between the signal pin and ground.

6. Selection Criteria for the Components

- **Logic level:** Every module connected to a GPIO pin must be compatible with 3.3 V. The encoder and the OLED are supplied from the 3.3 V pin for this reason.
- **Interrupt capability:** The two encoder channels must be connected to pins that can generate interrupts. On the ESP32-C6 all GPIO pins have this ability.
- **LED current:** A resistor of a few hundred ohms keeps the LED current at a few milliamperes, which is bright enough indoors and well within the capability of a GPIO pin.
- **Availability and cost:** All parts are inexpensive, widely available and need no soldering, so that the project can be repeated easily.

3.2 SOFTWARE REQUIREMENTS

This section outlines the development environment and the libraries used to program the board. Table 3.2 lists the software items and their purpose.

Table 3.2 Software requirements

S.No	Item	Name / Version	Purpose
1	Development environment	Arduino IDE 2.x with the ESP32 board package	Writing, compiling and uploading the program
2	Programming language	C/C++ (Arduino framework)	Control logic of the lock
3	Display library	U8g2	Drawing text and lines on the OLED [4]
4	Bus library	SPI.h (part of the core)	Hardware SPI for the display
5	Interrupt functions	attachInterrupt(), IRAM_ATTR	Run the encoder decoder on every edge of CLK and DT [5]
6	Tools	Serial Monitor and Serial Plotter (115200 baud)	Telemetry of the state, encoder value, slot and failures

The function attachInterrupt() registers a routine that runs whenever the level of a pin changes. The routine is declared with the IRAM_ATTR attribute so that it is stored in internal RAM and can run even when the flash memory is busy [1]. The variables that are shared between the interrupt routine and the main loop are declared volatile so that the compiler always reads their current value from memory. The function digitalRead() is used to sample the encoder pins, digitalWrite() drives the LEDs and the buzzer, and millis() provides the time base for the blinking alarm without any blocking delay.

Setting Up the Software

- Install the Arduino IDE and add the ESP32 board package from the Boards Manager.
- Install the U8g2 library from the Library Manager.
- Select the ESP32C6 Dev Module board and the correct serial port from the Tools menu.
- Paste the program of Section 5.2 and press Verify to check that it compiles without errors.
- Press Upload and open the Serial Monitor at 115200 baud to view the telemetry.

The Serial Plotter of the IDE can also be used. Plotting the encoder value together with the slot number and the failure count shows the behaviour of the debouncing logic as a graph while the dial is rotated quickly.

CHAPTER 4
SYSTEM DESIGN

4.1 BLOCK DIAGRAM

Figure 4.1 shows the overall block diagram of the Safecracker lock. The design is divided into four functional parts: the input, the controller, the output devices and the power supply.

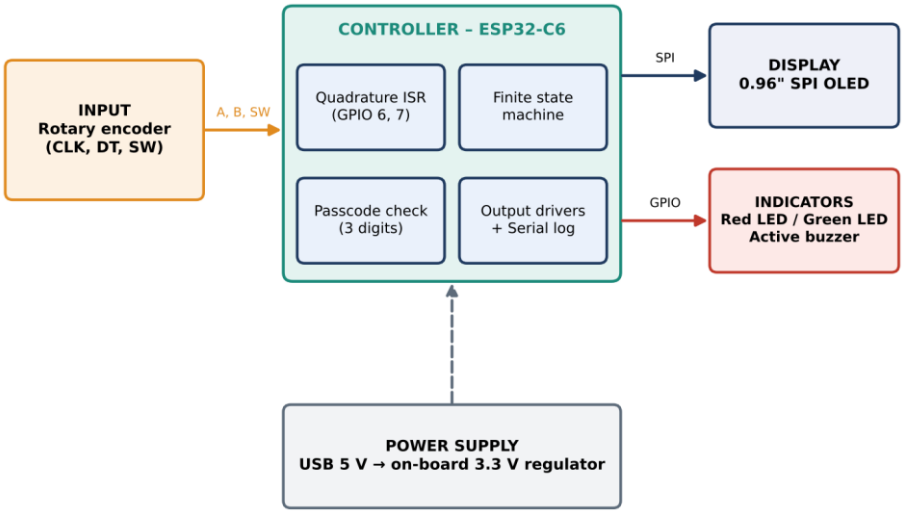


Figure 4.1 Block diagram of the proposed system

The input block is the rotary encoder with its push button. The controller block is the ESP32-C6. Inside it, the encoder interrupt service routine updates the dial counter, the finite state machine decides what the system does, the passcode checker compares the entry with the stored code, and the output drivers update the display, the LEDs and the buzzer and send the telemetry to the computer. The output block consists of the SPI OLED display, the red and the green LED, and the buzzer. The power supply block is the USB connection, which supplies 5 V to the board; the on-board regulator generates the 3.3 V used by the logic and by the modules.

Design Considerations and Calculations

The strength and the behaviour of the lock follow from a few simple numbers. The dial can take 16 values, 0 to 15, and the passcode has three digits, so the number of different combinations is

$$N = M^k = 16^3 = 4096 \quad (4.1)$$

where M is the number of dial positions and k is the number of digits. After three wrong attempts the system enters the alarm mode, so a person who guesses at random has A = 3 attempts before the lockout. The probability of opening the lock by chance in one lockout cycle is

$$P = A / N = 3 / 4096 \approx 0.073\ \% \tag{4.2}$$

Because the failure counter is held in volatile memory, an attacker with physical access could restart the board after every lockout. Even then, covering all combinations would need up to $\lceil 4096 / 3 \rceil = 1366$ restarts, which shows the order of magnitude of the effort. Section 9.2 proposes a stored lockout timer that removes this weakness.

The current of each status LED follows from Ohm's law. For a supply of 3.3 V from the GPIO pin, a forward voltage of about 2.0 V for a red LED and a series resistor of 220 Ω, the current is

$$I = (V_GPIO - V_F) / R = (3.3 - 2.0) / 220 \approx 5.9\ \text{mA} \tag{4.3}$$

A green LED has a slightly higher forward voltage of about 2.1 V to 2.2 V and therefore draws about 5 mA. These values are typical assumptions; the exact forward voltage should be taken from the datasheet of the LED that is used. The currents are small enough for a GPIO pin and give a clearly visible light.

Finally, the decoding of the encoder uses a 4-bit number, so 16 values are possible for each pair of old and new states. Of these, four belong to the clockwise sequence and four to the counter-clockwise sequence; the other eight are either repeated states or impossible jumps and are ignored (Table 6.1). Hence only half of the possible values of the 4-bit number can ever change the counter, which is the reason why random bounce does not disturb the count.

4.2 CIRCUIT DIAGRAM

Figure 4.2 shows the circuit connections of the system, and Table 4.1 lists the connections between the board and the modules. The encoder is connected to GPIO 6, 7 and 10, the OLED to the SPI pins GPIO 21, 22, 8, 9 and 1, the buzzer to GPIO 19, the red LED to GPIO 18 and the green LED to GPIO 20.

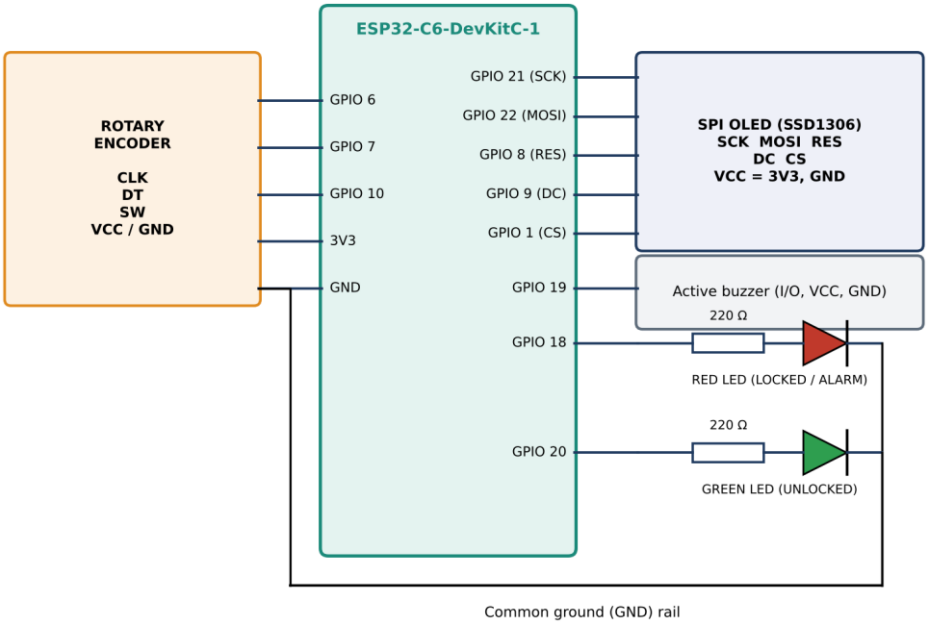


Figure 4.2 Circuit diagram of the proposed system

Table 4.1 Pin connections of the ESP32-C6 with the modules

Module	Module pin	ESP32-C6 pin	Signal type / Function
Rotary encoder	CLK	GPIO 6	Interrupt input (phase A)
Rotary encoder	DT	GPIO 7	Interrupt input (phase B)
Rotary encoder	SW	GPIO 10	Push button (internal pull-up)
Rotary encoder	VCC / GND	3.3 V / GND	Logic power
OLED (SPI)	SCK	GPIO 21	SPI clock
OLED (SPI)	SDA (MOSI)	GPIO 22	SPI data
OLED (SPI)	RES	GPIO 8	Reset
OLED (SPI)	DC	GPIO 9	Data / command
OLED (SPI)	CS	GPIO 1	Chip select
OLED (SPI)	VCC / GND	3.3 V / GND	Logic power
Red LED	Anode via 220 Ω	GPIO 18	Output, active HIGH (locked / alarm)
Green LED	Anode via 220 Ω	GPIO 20	Output, active HIGH (unlocked)
Both LEDs	Cathode	GND	Common ground
Active buzzer	I/O	GPIO 19	Output, active HIGH
Active buzzer	VCC / GND	3.3 V / GND	Power (3-pin module)

The two LEDs and their resistors replace the LED ring of the original design. Since the ring is no longer used, the 5 V rail of the board is not needed at all: every part of the

circuit is supplied at 3.3 V, and the only 5 V source is the USB connector that feeds the board itself.

Important Wiring Precautions

- **Observe LED polarity:** The long lead (anode) goes to the resistor and the short lead (cathode) to ground. A reversed LED does not light.
- **Never omit the resistors:** Without a series resistor the LED and the GPIO pin can be damaged.
- **Use 3.3 V only:** The GPIO pins of the ESP32-C6 are not designed for 5 V signals, so the encoder, the OLED and the buzzer must be powered from the 3.3 V pin.
- **Common ground:** All modules and both LEDs must share the ground of the board.
- **Check the OLED variant:** A seven-pin module is an SPI display, a four-pin module is an I2C display; the wiring of Table 4.1 applies to the first type only.

Layout Considerations

The wires of the encoder carry fast logic edges and should be kept short so that they do not pick up noise that could look like an extra step. The LEDs and the buzzer can be placed close to the display so that the operator sees all the indications at once. If the encoder is unstable, a ceramic capacitor of about 10 nF between each channel and ground can be added; with the validity check of the software it is normally not needed.

4.3 STATE MACHINE AND FLOWCHART

The behaviour of the lock is described by the state diagram of Figure 4.3 and by Table 4.2. The system starts in LOCKED. In this state the operator dials the three digits. If the three digits are correct, the machine moves to SUCCESS. If they are wrong, the failure counter increases and the entry starts again, unless it was the third failure, in which case the machine moves to ALARM.

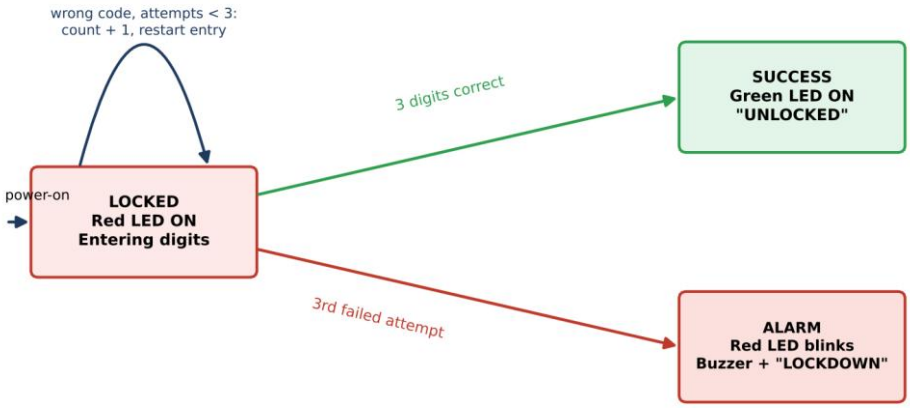


Figure 4.3 State diagram of the security state machine

Table 4.2 States of the security state machine

State	Entered when	Outputs	Leaves when
LOCKED	Power-on, or after a wrong code with fewer than 3 failures	Red LED on, green LED off; OLED secure terminal; click per step	3 digits correct → SUCCESS. 3rd failure → ALARM
SUCCESS	Correct passcode entered	Green LED on, red LED off; OLED shows UNLOCKED	Only by resetting the board
ALARM	Third failed attempt	Red LED blinks, buzzer pulses every 200 ms; OLED shows LOCKDOWN	Only by resetting the board

Figure 4.4 shows the program flow. The program has an initialisation part that runs once and a loop that repeats indefinitely. In the initialisation the pins are defined, the encoder inputs are configured with pull-up resistors, the two interrupts are attached, and the display and the LEDs are started. The loop executes the code of the present state.

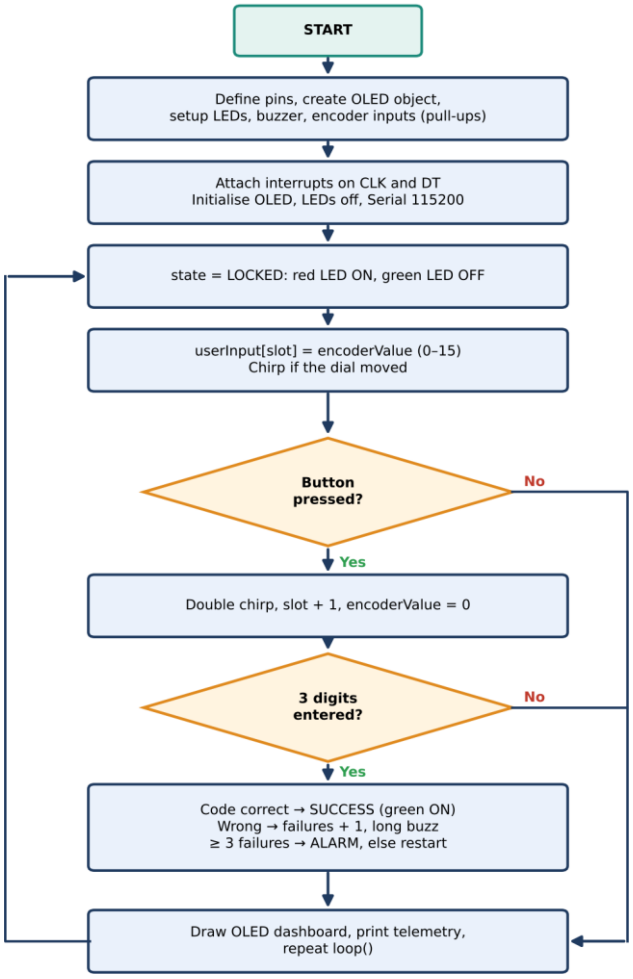


Figure 4.4 Flowchart of the system operation

Algorithm

- **Step 1:** Start. Define the pins, create the display object and set currentState = LOCKED.
- **Step 2:** Configure the buzzer and the LEDs as outputs and the encoder pins as inputs with pull-ups. Attach the interrupts to CLK and DT. Initialise the display and switch both LEDs off.
- **Step 3:** In the interrupt: form movementState from the previous and the present phase; increment the counter for a clockwise sequence, decrement it for a counter-clockwise sequence, wrap between 0 and 15, and save the present phase.
- **Step 4:** In LOCKED: switch the red LED on and the green LED off, copy encoderValue into userInput[currentDigitIndex] and chirp if the value changed.
- **Step 5:** If the button is pressed: confirm with a double chirp, increase the slot index and reset the dial to 0.
- **Step 6:** After the third digit compare the entry with the passcode. If it is equal, go to SUCCESS; otherwise increase the failure count, sound the buzzer and either restart the entry or go to ALARM.
- **Step 7:** Draw the OLED interface, print the telemetry and repeat from Step 3 for as long as power is applied.

CHAPTER 5
IMPLEMENTATION

This chapter details the construction of the prototype and the program that runs on the ESP32-C6. The system was realised by wiring the encoder, the OLED, the buzzer and the two LEDs to the board as in the circuit diagram, and by writing a C++ program that implements the interrupt decoder and the three-state machine.

5.1 HARDWARE IMPLEMENTATION

The prototype was assembled on a solderless breadboard in the following steps.

- **Board and power rails:** The ESP32-C6 board is placed on the breadboard and the 3.3 V and GND pins are connected to the power rails of the board.
- **Rotary encoder:** The encoder is placed on the breadboard (a small piece of double-sided tape holds it in place). CLK is wired to GPIO 6, DT to GPIO 7 and SW to GPIO 10, and VCC and GND to the rails.
- **OLED display:** SCK is wired to GPIO 21, SDA to GPIO 22, RES to GPIO 8, DC to GPIO 9 and CS to GPIO 1, and VCC and GND to the rails.
- **Buzzer:** The signal pin of the buzzer is wired to GPIO 19 and its supply pin and ground to the rails.
- **Red LED:** The anode is connected through a 220 Ω resistor to GPIO 18 and the cathode to ground.
- **Green LED:** The anode is connected through a 220 Ω resistor to GPIO 20 and the cathode to ground.
- **Power-up:** The board is connected to the computer by the USB cable only after all the connections have been checked.

Pre-power Checklist

Before the USB cable is connected the following points were checked: no wire connects 3.3 V directly to GND; the polarity of both LEDs is correct and each has its resistor; the encoder, the OLED and the buzzer are supplied from the 3.3 V pin and not from 5 V; and the pin numbers of every wire agree with Table 4.1.

Safety Precautions

- Never connect a 5 V source to a GPIO pin of the ESP32-C6.
- Disconnect the USB cable before changing the wiring.
- Do not remove the series resistors of the LEDs.
- Keep the buzzer away from the ear during the alarm test, since it sounds continuously.

Sequence of Hardware Testing

The hardware is tested in stages so that any fault can be located easily. First the two LEDs are tested with a short sketch that switches them on one after the other. Next the encoder is tested by printing the counter on the serial monitor. Then the display is tested with a text message, and finally the buzzer is tested with a short beep. Only when every part works alone is the complete program uploaded.

5.2 SOFTWARE IMPLEMENTATION

The program was written in the Arduino IDE in C++ and uploaded through the USB cable. It follows the flowchart of Figure 4.4 and uses the U8g2 library for the display.

1. Pin Definitions

Constants with meaningful names are used for every pin: ENCODER_CLK, ENCODER_DT and ENCODER_SW for the encoder, SPI_OLED_SCK, SPI_OLED_SDA, SPI_OLED_CS, SPI_OLED_RES and SPI_OLED_DC for the display, LED_RED_PIN and LED_GREEN_PIN for the two status LEDs, and BUZZER_PIN for the buzzer. Changing a pin in the program then needs a change in one line only.

2. Encoder Interrupt Service Routine

The routine readEncoder() is attached to both CLK and DT with the CHANGE mode, so it runs on every edge of either channel. It reads the two pins and forms the 2-bit number currentPhase = (A << 1) | B. The previous phase is shifted two places to the left and combined with the present phase to give the 4-bit number movementState. Four values of movementState belong to a clockwise step (0b1101, 0b0100, 0b0010, 0b1011) and four to a counter-clockwise step (0b1110, 0b0111, 0b0001, 0b1000). For any other value the counter is not changed, but the present phase is still saved. The counter wraps from 15 to 0 and from 0 to 15, which gives a circular dial of 16 positions.

3. Helper Functions

The function setLeds(redOn, greenOn) switches the two LEDs in a single call and is used by every state, so the rule that the red and the green LED are never on together is kept in one place. The function soundChirp() produces a click of 40 ms on the buzzer.

4. Setup and Loop

In setup() the serial port is opened at 115200 baud, the buzzer and the LED pins are set as outputs, the encoder pins are set as inputs with pull-ups, the two interrupts are attached, the SPI bus and the OLED are started, and both LEDs are switched off. In loop() a switch statement executes the code of the present state. The LOCKED branch keeps the red LED on, copies the dial value into the entry array, sounds a click when the value changes, handles the push button, checks the code after the third digit and draws the OLED interface. The SUCCESS branch lights the green LED and shows UNLOCKED. The

ALARM branch blinks the red LED and the buzzer with the expression (millis() / 200) % 2, which toggles every 200 ms without blocking.

5. Complete Program

The complete program of the lock is given below.

```
#include <Arduino.h>
#include <SPI.h>
#include <U8g2lib.h>

// --- Pin definitions ---
#define SPI_OLED_SCK  21
#define SPI_OLED_SDA  22
#define SPI_OLED_CS    1
#define SPI_OLED_RES   8
#define SPI_OLED_DC    9

#define ENCODER_CLK    6
#define ENCODER_DT     7
#define ENCODER_SW    10

#define LED_RED_PIN   18   // LOCKED / ALARM indicator
#define LED_GREEN_PIN 20   // UNLOCKED indicator
#define BUZZER_PIN    19

// --- Display object ---
U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI oled(U8G2_R0, SPI_OLED_CS,
                                            SPI_OLED_DC, SPI_OLED_RES);

// --- Security state machine ---
enum SystemState { LOCKED, SUCCESS, ALARM };
SystemState currentState = LOCKED;

const int PASSCODE[3] = {9, 1, 15};   // change the passcode here
int userInput[3] = {0, 0, 0};
int currentDigitIndex = 0;
int failedAttempts = 0;

// --- Encoder tracking (shared with the ISR) ---
volatile int encoderValue = 0;
volatile uint8_t previousPhase = 0;
int lastEncoderValue = 0;

// --- Bounce-proof interrupt service routine ---
void IRAM_ATTR readEncoder() {
  uint8_t currentA = digitalRead(ENCODER_CLK);
  uint8_t currentB = digitalRead(ENCODER_DT);

  uint8_t currentPhase  = (currentA << 1) | currentB;
  uint8_t movementState = (previousPhase << 2) | currentPhase;

  // valid clockwise sequence
  if (movementState == 0b1101 || movementState == 0b0100 ||
      movementState == 0b0010 || movementState == 0b1011) {
    encoderValue++;
    if (encoderValue > 15) encoderValue = 0;
  }
  // valid counter-clockwise sequence
  else if (movementState == 0b1110 || movementState == 0b0111 ||
           movementState == 0b0001 || movementState == 0b1000) {
    encoderValue--;
    if (encoderValue < 0) encoderValue = 15;
  }
  previousPhase = currentPhase;
}
```

```
// --- Helper functions ---
void setLeds(bool redOn, bool greenOn) {
  digitalWrite(LED_RED_PIN,   redOn   ? HIGH : LOW);
  digitalWrite(LED_GREEN_PIN, greenOn ? HIGH : LOW);
}

void soundChirp() {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(40);
  digitalWrite(BUZZER_PIN, LOW);
}

void setup() {
  Serial.begin(115200);

  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_RED_PIN, OUTPUT);
  pinMode(LED_GREEN_PIN, OUTPUT);
  pinMode(ENCODER_SW, INPUT_PULLUP);
  pinMode(ENCODER_CLK, INPUT_PULLUP);
  pinMode(ENCODER_DT, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(ENCODER_CLK), readEncoder, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENCODER_DT), readEncoder, CHANGE);

  SPI.begin(SPI_OLED_SCK, -1, SPI_OLED_SDA, SPI_OLED_CS);
  oled.begin();

  setLeds(false, false);
}

void loop() {
  unsigned long currentMillis = millis();

  switch (currentState) {

    case LOCKED: {
      setLeds(true, false);                 // red ON, green OFF
      userInput[currentDigitIndex] = encoderValue;

      // click when the dial moves
      if (encoderValue != lastEncoderValue) {
        soundChirp();
        lastEncoderValue = encoderValue;
      }

      // button pressed: confirm the digit
      if (digitalRead(ENCODER_SW) == LOW) {
        delay(50);                          // ignore button bounce
        while (digitalRead(ENCODER_SW) == LOW);   // wait for release

        soundChirp(); delay(50); soundChirp();

        currentDigitIndex++;
        encoderValue = 0;                   // dial back to 0

        if (currentDigitIndex >= 3) {       // verification
          if (userInput[0] == PASSCODE[0] && userInput[1] == PASSCODE[1] &&
              userInput[2] == PASSCODE[2]) {
            currentState = SUCCESS;
          } else {
            failedAttempts++;
            digitalWrite(BUZZER_PIN, HIGH);
            delay(500);
            digitalWrite(BUZZER_PIN, LOW);
```

```
        if (failedAttempts >= 3) {
          currentState = ALARM;
        } else {
          currentDigitIndex = 0;        // try again
        }
      }
    }
  }

  // OLED dashboard
  oled.clearBuffer();
  oled.setFont(u8g2_font_6x10_tf);
  oled.setCursor(18, 12); oled.print("SECURE TERMINAL");
  oled.drawHLine(0, 16, 128);

  oled.setFont(u8g2_font_logisoso22_tr);
  if (currentDigitIndex > 0) oled.drawStr(20, 48, "*");
  else { oled.setCursor(20, 48); oled.print(userInput[0]); }

  if (currentDigitIndex > 1) oled.drawStr(58, 48, "*");
  else if (currentDigitIndex == 1) {
    oled.setCursor(58, 48); oled.print(userInput[1]);
  }
  else oled.drawStr(58, 48, "-");

  if (currentDigitIndex > 2) oled.drawStr(96, 48, "*");
  else if (currentDigitIndex == 2) {
    oled.setCursor(96, 48); oled.print(userInput[2]);
  }
  else oled.drawStr(96, 48, "-");

  if (currentDigitIndex == 0) oled.drawHLine(18, 52, 20);
  if (currentDigitIndex == 1) oled.drawHLine(56, 52, 20);
  if (currentDigitIndex == 2) oled.drawHLine(94, 52, 20);
  oled.sendBuffer();

  // telemetry
  Serial.print("State:");    Serial.print(currentState);      Serial.print(",");
  Serial.print("Encoder:");  Serial.print(encoderValue);      Serial.print(",");
  Serial.print("Slot:");     Serial.print(currentDigitIndex); Serial.print(",");
  Serial.print("Failures:"); Serial.println(failedAttempts);
  break;
}

case SUCCESS: {
  setLeds(false, true);                 // green ON, red OFF
  oled.clearBuffer();
  oled.setFont(u8g2_font_9x15B_tf);
  oled.setCursor(22, 38);
  oled.print("UNLOCKED");
  oled.sendBuffer();
  Serial.println("State:SUCCESS_UNLOCKED");
  break;
}

case ALARM: {
  if ((currentMillis / 200) % 2 == 0) { // blink every 200 ms
    setLeds(true, false);
    digitalWrite(BUZZER_PIN, HIGH);
  } else {
    setLeds(false, false);
    digitalWrite(BUZZER_PIN, LOW);
  }
  oled.clearBuffer();
  oled.setFont(u8g2_font_9x15B_tf);
  oled.setCursor(30, 38);
  oled.print("LOCKDOWN");
  oled.sendBuffer();
```

```
      Serial.println("State:ALARM_LOCKDOWN");
      break;
    }
  }
}
```

6. Changes for the I2C Version of the Display

If the OLED module has only four pins (GND, VCC, SCL, SDA), it uses the I2C bus. The wiring is simpler: SDA is connected to GPIO 4 and SCL to GPIO 5, and the pins RES, DC and CS are no longer needed. In the program, the SPI pin definitions are replaced by I2C_SDA = 4 and I2C_SCL = 5, the line #include <Wire.h> is added, the display object is created with U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE), and in setup() the call SPI.begin(...) is replaced by Wire.begin(I2C_SDA, I2C_SCL). The rest of the program, including the state machine and the LED control, is unchanged.

7. Common Problems and Remedies

Table 5.1 lists the problems that are most often met when the circuit is built for the first time, together with their probable causes and remedies.

Table 5.1 Common problems and remedies

Symptom	Probable cause	Remedy
A LED does not light	LED reversed, wrong pin, or missing resistor connection	Check the polarity and the GPIO number (18 red, 20 green)
Display stays blank	Wrong OLED type (SPI/I2C) or a loose wire	Check the number of pins on the module and the pin table
Dial value jumps or skips	Loose encoder pin, noisy wires, or wrong pins	Shorten the wires and recheck GPIO 6 and 7
Value changes by 4 per click	Encoder produces four legal steps per detent	Divide the count by 4 or accept every 4th step (Section 7.3)
Dial turns the wrong way	CLK and DT interchanged	Swap the two wires or exchange the sequences in the ISR

CHAPTER 6
WORKING OF THE SYSTEM

This chapter explains the operation of the Safecracker lock step by step. The system works as a continuous loop: the operator turns the dial, the interrupt routine updates the counter, the main loop shows the value and handles the button, and the state machine decides what the lock does after the third digit.

1. System Initialisation

When power is applied, the board boots and runs setup() once. The serial port is opened at 115200 baud, the buzzer and the LED pins are set as outputs, and the encoder pins are set as inputs with pull-up resistors. The two interrupts are attached, the OLED is started and both LEDs are switched off. The state is LOCKED, the digit index is 0, the failure counter is 0 and the dial counter is 0. As soon as loop() starts, the red LED is switched on and the OLED shows the secure-terminal screen with the digit 0 in the first slot.

2. Reading the Dial

Whenever the encoder shaft moves, one of the two channels changes level and an interrupt is generated. The routine reads both pins, forms the 4-bit movementState and looks it up in the decoding table shown in Table 6.1. Only the legal clockwise or counter-clockwise combinations change the counter. A repeated state, which can happen when a contact bounces, and an impossible jump, in which both channels seem to change at once, are ignored. This is the debouncing action of the program: no waiting is needed, and a bounce on one edge cannot produce a step unless the old and the new states form a legal pair.

Table 6.1 Decoding of movementState (previous phase and present phase)

movementState	Previous → present (A B)	Action on the counter	Meaning
0b0010	00 → 10	+1	Clockwise step
0b1011	10 → 11	+1	Clockwise step
0b1101	11 → 01	+1	Clockwise step
0b0100	01 → 00	+1	Clockwise step
0b0001	00 → 01	−1	Counter-clockwise step
0b0111	01 → 11	−1	Counter-clockwise step
0b1110	11 → 10	−1	Counter-clockwise step
0b1000	10 → 00	−1	Counter-clockwise step
0b0000, 0b0101, 0b1010, 0b1111	No change in the state	None	Repeated state, ignored

movementState	Previous → present (A B)	Action on the counter	Meaning
0b0011, 0b0110, 0b1001, 0b1100	Both channels changed	None	Impossible jump (noise), ignored

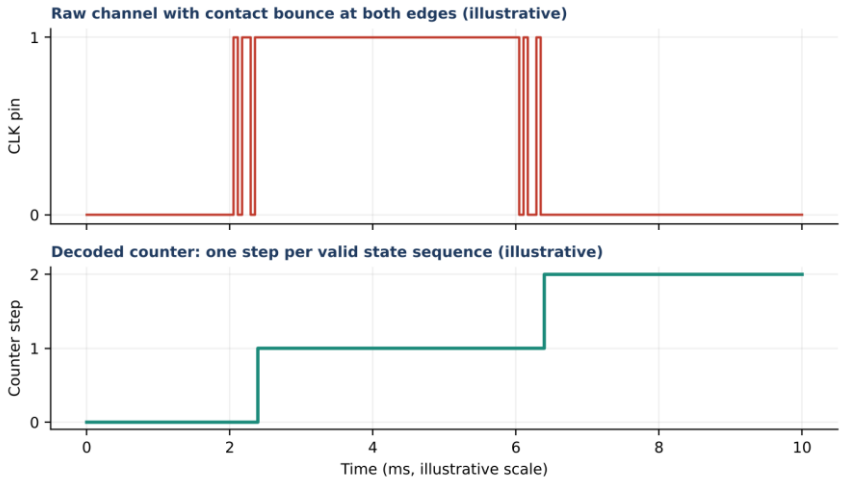


Figure 6.1 Illustrative rejection of contact bounce by the state-tracking decoder

Figure 6.1 illustrates the idea. The upper trace shows a channel that flickers at both edges, as a worn contact does. The lower trace shows that the decoded counter advances by exactly one step for each real movement. The figure is a schematic illustration of the principle and not a measured oscilloscope trace.

3. Showing the Digit and Giving Feedback

In every pass of the loop the dial counter is copied into userInput[currentDigitIndex], so the OLED always shows the digit that is being dialled in the active slot, and an underline marks that slot. If the counter has changed since the last pass, the buzzer gives a short click of 40 ms, which tells the operator that the dial has moved. Digits that have already been entered are replaced by an asterisk, so that a person looking at the screen cannot read the code.

4. Confirming a Digit

When the operator presses the push button, the program waits 50 ms to ignore the bounce of the button, waits until the button is released, gives a double chirp, increases the digit index and resets the dial counter to 0, so that every digit starts from 0. The underline on the display moves to the next slot.

5. Verification of the Passcode

After the third digit has been confirmed, the three stored values are compared one by one with the passcode array. If all three agree, the state changes to SUCCESS. If any value differs, the failure counter is increased and the buzzer sounds for 500 ms as an error

signal. When the counter is below 3 the digit index returns to 0 and the operator may try again; when it reaches 3 the state changes to ALARM.

6. Indications in the Three States

Table 6.2 summarises what the operator sees and hears in each state.

Table 6.2 Outputs of the lock in each state

State	Red LED	Green LED	Buzzer	OLED
LOCKED	On (steady)	Off	Click per step; double chirp per digit; 0.5 s tone on a wrong code	SECURE TERMINAL, three slots, underline
SUCCESS	Off	On (steady)	Silent	UNLOCKED
ALARM	Blinks (200 ms)	Off	Pulses with the red LED	LOCKDOWN

The two LEDs are never on at the same time. A person who walks up to the lock can therefore tell its condition from the colour alone: red means that access is denied and green means that the lock is open.

7. Worked Example

Assume the passcode {9, 1, 15} that is stored in the program. Table 6.3 follows an operator who enters the correct code. The dial is turned to 9, confirmed, turned to 1, confirmed, turned to 15 and confirmed.

Table 6.3 Entry of the correct passcode {9, 1, 15}

Step	Operator action	Slot index	OLED shows	LEDs
1	Power on; no action	0	Digit 0 in slot 1	Red on
2	Turn the dial to 9	0	Digit 9 in slot 1	Red on
3	Press the button	1	* in slot 1; dial value 0 in slot 2	Red on
4	Turn the dial to 1 and press	2	* *; dial value 0 in slot 3	Red on
5	Turn the dial to 15 and press	3	UNLOCKED	Green on, red off

If the third digit were 14 instead of 15, the comparison would fail. The failure counter would become 1, the buzzer would sound for half a second and the entry would restart from slot 1 with the red LED still on. After three such failures the red LED would start to blink, the buzzer would pulse, and the display would show LOCKDOWN until the board is reset.

8. Continuous Monitoring

In each pass of the loop the program sends one line to the serial port in the form State:0,Encoder:9,Slot:0,Failures:0. The state is printed as a number (0 = LOCKED, 1 = SUCCESS, 2 = ALARM). The Serial Plotter can draw the encoder value, the slot and the failure count against time, which shows the steps of the dial and the progress through the three digits.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 EXPERIMENTAL SETUP

The experiment was carried out on the breadboard prototype described in Chapter 5. The ESP32-C6 board was connected to a computer through the USB cable so that the serial monitor could display the telemetry. The encoder, the SPI OLED, the buzzer, the red LED and the green LED were wired as in Table 4.1. A digital multimeter can be used to check the 3.3 V supply and the voltage across the LED resistors.

Test Procedure

- Upload the program of Section 5.2 and open the serial monitor at 115200 baud.
- Turn the dial slowly clockwise and counter-clockwise and confirm that the value on the display and in the telemetry changes in the correct direction and wraps between 0 and 15.
- Turn the dial quickly in both directions and observe on the Serial Plotter that the encoder value changes without spurious jumps.
- Enter the correct passcode and note the state, the LEDs and the display.
- Reset the board and enter a wrong code three times, noting the failure count, the buzzer, the LEDs and the display after each attempt.

7.2 OBSERVED RESULTS

Table 7.1 gives the behaviour that the design predicts for each stage of operation, and the column on the right is reserved for the observation recorded on the prototype. Table 7.2 lists the functional tests that were planned for the prototype, again with a column for the observation.

Table 7.1 Expected behaviour of the lock at each stage

Stage	Expected display and LEDs	Expected sound	Observed
Power-on	SECURE TERMINAL, digit 0; red on, green off	Silent	
Dial moved	Digit changes by 1 per step (0–15, wraps)	Click of 40 ms	
Digit confirmed	* in the confirmed slot; dial resets to 0	Double chirp	
Wrong code (1st, 2nd)	Entry restarts at slot 1; red stays on	Tone of 0.5 s	
Wrong code (3rd)	LOCKDOWN; red blinks	Pulsing alarm	
Correct code	UNLOCKED; green on, red off	Silent	

Table 7.2 Functional tests of the lock

S.No	Test condition	Expected behaviour	Observation
1	Slow clockwise rotation	Value increases; wraps from 15 to 0	
2	Slow counter-clockwise rotation	Value decreases; wraps from 0 to 15	
3	Fast rotation in both directions	No spurious jumps in the Serial Plotter	
4	Correct code {9, 1, 15}	SUCCESS: green on, red off, UNLOCKED	
5	Wrong code once	Failures = 1; entry restarts; red on	
6	Wrong code three times	ALARM: red blinks, buzzer pulses, LOCKDOWN	
7	Passcode changed in the program	New code opens the lock; old code fails	
8	Reset after ALARM or SUCCESS	Returns to LOCKED with red LED on	

The strength of the lock can be calculated from Equations (4.1) and (4.2). Table 7.3 gives the number of combinations and the probability of opening the lock by chance within the three allowed attempts for passcodes of different length, using the same dial of 16 positions. Figure 7.1 shows the same values on a logarithmic scale.

Table 7.3 Combination space and chance of guessing for a 16-position dial

Passcode length k	Combinations N = 16^k	Attempts allowed	Chance of success (3 / N)
1 digit	16	3	18.75 %
2 digits	256	3	1.17 %
3 digits (this project)	4096	3	0.073 %
4 digits	65 536	3	0.0046 %

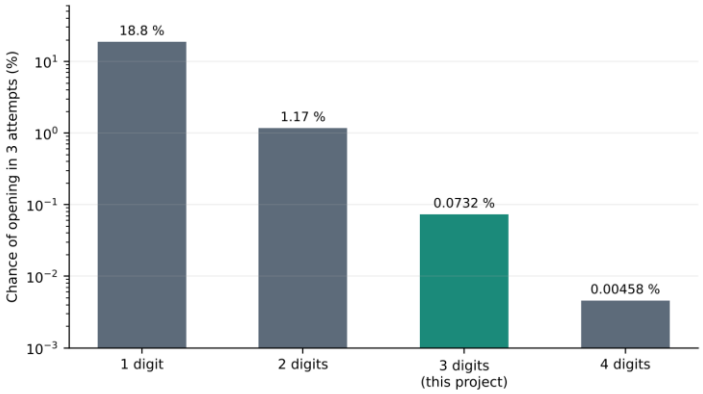


Figure 7.1 Chance of opening the lock by guessing within three attempts

7.3 DISCUSSION

1. Reliability of the Dial Input

The decoder accepts a step only when the previous and the present states form one of the eight legal pairs of Table 6.1. A bounce that returns the channel to its previous value produces a repeated state and is ignored, and a spurious pulse on one channel cannot advance the counter unless it is followed by the correct transition on the other. In this way the debouncing is done by logic and not by waiting, so the program remains free to refresh the display and to run the state machine.

2. Steps per Detent

Every legal transition changes the counter by one, and a complete cycle of the quadrature signals contains four legal transitions. Many mechanical encoders produce one complete cycle per detent (click) of the knob; with such an encoder the displayed value would change by four for each click and a full turn would cover the 16 positions in only four clicks. Other encoders are built so that the contacts complete a cycle only after several clicks. The number of counts per click therefore depends on the encoder in use and should be checked first on the serial monitor. If four counts per click are observed, the usual remedy is to change the counter only on every fourth legal transition, or to divide the value by 4 before it is displayed.

3. Two LEDs and Blocking Delays

The red and the green LED show only the state of the lock, while the position of the dial is shown as a number on the OLED. This simplifies the circuit: two GPIO pins and two resistors are enough, the 5 V rail is not needed, and each LED draws only about 5 mA. Short delays remain in the loop (40 ms per chirp, 50 ms for the button and 500 ms for the error tone); the display pauses during them, but the interrupt keeps updating the counter, so no dial movement is lost.

4. Security Strength

With 16 positions and three digits there are 4096 combinations, and the chance of opening by a random guess in three attempts is about 0.073 % (Table 7.3). This is adequate for a demonstration, but it is not a real security level. The passcode is stored in plain text in the program, the failure counter is lost when the board is reset, and the active digit is shown on the display while it is being dialled. For these reasons the system must be regarded as a learning prototype.

CHAPTER 8
ADVANTAGES, LIMITATIONS AND APPLICATIONS

8.1 ADVANTAGES

- **Simple and Low Cost:** The system needs only an ESP32-C6 board, an encoder, a small OLED, a buzzer, two LEDs and two resistors.
- **Reliable Input:** Interrupt-based, state-tracking decoding follows every movement of the dial and ignores contact bounce without any waiting.
- **Clear Status Indication:** The red and the green LED show the condition of the lock at a glance, and the OLED and the buzzer give detailed feedback.
- **Defined Behaviour:** The finite state machine gives clear rules for a correct code, a wrong code and repeated wrong codes.
- **Attempt Limit:** The lockout after three failures prevents unlimited guessing.
- **Easy to Understand and Modify:** The program is short, uses named constants, and the passcode and the pins can be changed in one line.
- **Expandable Design:** Wireless connectivity, a real bolt or non-volatile storage can be added to the same board without changing the basic circuit.

8.2 LIMITATIONS

- **Hard-coded Passcode:** The code can be changed only by editing and uploading the program.
- **Volatile Failure Counter:** The counter is lost when the board is reset, so a lockout can be bypassed by restarting.
- **Terminal States:** SUCCESS and ALARM can be left only by resetting the board.
- **Blocking Delays:** The chirps, the button debounce and the error tone use delay(), which pauses the display for short periods.
- **Status Only:** The LEDs show the state but not the dial position.
- **No Physical Locking:** The prototype does not drive a bolt, solenoid or servo.

8.3 APPLICATIONS

- **Digital Safes and Lockers:** Dial-operated electronic safes and storage lockers.
- **Access Control Prototypes:** Laboratory models of door or cabinet controllers.
- **Industrial Selector Switches:** Rotary selection of modes or recipes with a confirmation button.
- **Automotive and Consumer Interfaces:** Control knobs similar to the rotary controllers of car infotainment systems.
- **Educational Laboratories:** Teaching interrupts, quadrature decoding, finite state machines and embedded interfaces.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 CONCLUSION

The Project Safecracker lock showed that a secure-looking combination lock can be built from a small number of low-cost parts and a short program. A quadrature rotary encoder serves as the dial, and an interrupt service routine with bitwise state tracking converts its two phase-shifted signals into a clean 16-position counter without any blocking delay. A three-state machine with the modes LOCKED, SUCCESS and ALARM compares the three-digit entry with the stored passcode and locks the system after three failed attempts. An SPI OLED shows the secure-terminal interface, an active buzzer gives audible feedback, and a red and a green LED show the condition of the lock clearly.

The relations between the dial positions, the number of combinations and the chance of a lucky guess were worked out and tabulated, and the practical requirements of the design were recorded: the use of 3.3 V logic for every module, the series resistors of the LEDs, the pull-up resistors of the encoder, the IRAM_ATTR attribute of the interrupt routine, and the selection of the correct OLED type.

The system is simple, inexpensive and easy to build and to modify. Its main limitations are the hard-coded passcode, the volatile failure counter and the short blocking delays, which the improvements below can remove. The project also gave practical experience in reading datasheets, wiring a mixed circuit and debugging interrupt-driven code.

9.2 FUTURE SCOPE

The prototype demonstrates the basic principle, and a number of enhancements can make it more capable and more secure:

- **Master-key Reset and Stored Passcode:** Holding the encoder button for three seconds in the SUCCESS state could start the entry of a new three-digit code that is saved in non-volatile storage with the Preferences library, so that the code survives a power cycle.
- **Timed Lockout:** The ALARM state could use millis() to impose a lockout of 60 seconds, shown as a countdown on the OLED, after which the machine returns to LOCKED instead of waiting for a reset. The failure counter should also be stored in non-volatile memory.
- **Richer Sound:** A passive piezo transducer driven with ledcWrite() could give a 1500 Hz click on every step and distinct tone sequences for success and failure.
- **Non-blocking Program and Longer Codes:** Replacing delay() with millis()-based timers and using more digits, or a stored hash of the code, would improve responsiveness and security.

- **Physical Bolt and Wireless Alerts:** A solenoid or servo could release a real latch in the SUCCESS state, and the Wi-Fi or Bluetooth functions of the ESP32-C6 could send an alert when the alarm is raised.

REFERENCES

[1] Espressif Systems, “ESP32-C6 Series Datasheet,” Espressif Systems.

[2] Espressif Systems, “ESP32-C6-DevKitC-1 User Guide,” ESP-IDF Programming Guide, docs.espressif.com.

[3] Solomon Systech, “SSD1306: 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller,” datasheet.

[4] O. Kraus, “U8g2: Library for monochrome displays,” GitHub repository olikraus/u8g2.

[5] Arduino, “attachInterrupt(),” Arduino Language Reference, docs.arduino.cc.

[6] J. Ganssle, “A Guide to Debouncing,” The Ganssle Group.

[7] P. Horowitz and W. Hill, The Art of Electronics, 3rd ed., Cambridge University Press, 2015.

[8] J. F. Wakerly, Digital Design: Principles and Practices, Pearson Education.

[9] F. Gray, “Pulse code communication,” U.S. Patent 2,632,058, 1953.

[10] P. Stoffregen, “Encoder Library,” PJRC, pjrc.com.

[11] B. Buxton, “Rotary Encoder Library for Arduino,” GitHub repository buxtronix/arduino.

PO &PSO Attainment

POs	Graduate Attribute	Attainment	Justification
PO1	**Engineering Knowledge** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in respectively to develop to the solution of complex engineering problems.	**Yes**	Applied soil mechanics, foundation engineering, bearing capacity, settlement, shear strength and geotechnical engineering principles to develop project solutions.
PO2	**Problem Analysis** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development	**Yes**	Identified and analyzed geotechnical problems related to soil properties, foundation selection, bearing capacity, settlement and foundation performance.
PO3	**Design/Development of Solutions** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required	**Yes**	Developed suitable and sustainable foundation solutions considering soil conditions, structural requirements, safety, economy and environmental factors.
PO4	**Conduct Investigations of Complex Problems**	**Yes**	Conducted investigations using soil testing, field/laboratory data, case studies and

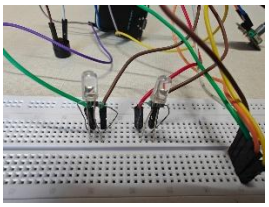
POs	Graduate Attribute	Attainment	Justification
	Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions		analysis to evaluate foundation performance and develop valid conclusions.
PO5	**Engineering Tool Usage** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems	**Yes**	Utilized engineering software, CAD tools, spreadsheets, analytical methods and AI-based tools for soil analysis, bearing capacity, settlement prediction and foundation-related studies.
PO6	**The Engineer and The World** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment	**Yes**	Considered environmental, societal, safety, economic and sustainability aspects while developing appropriate foundation solutions for real-world construction conditions.
PO7	**Ethics** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws	**Yes**	Followed responsible engineering practices by considering foundation safety, relevant codes and standards, environmental responsibility, data integrity and professional ethics.

POs	Graduate Attribute	Attainment	Justification
PO8	**Individual and Collaborative Team work** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.	**Yes**	Developed teamwork and leadership skills through group-based project activities involving site investigation, soil testing, analysis, design, prototype development and documentation.
PO9	**Communication** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.	**Yes**	Communicated project objectives, methodology, analysis, design outcomes and conclusions through project books, technical reports, presentations and project demonstrations.
PO10	**Project Management and Finance** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.	**Yes**	Applied project planning, task allocation, time management, resource utilization and cost considerations in the execution of Foundation Engineering projects.
PO11	**Life-Long Learning** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and	**Yes**	Developed lifelong learning and critical-thinking skills by exploring emerging technologies, AI-based geotechnical tools,

POs	Graduate Attribute	Attainment	Justification
	emerging technologies and iii) critical thinking in the broadest context of technological change		innovative foundation techniques and current engineering practices.

PSOs	Program Specific Outcomes (PSOs)	Attainment	Justification
PSO1	Application of Engineering knowledge – To analyze, design and develop solutions by applying the geo technical concepts for societal and industrial needs.	**Yes**	Applied soil mechanics and foundation engineering principles to analyze soil properties, bearing capacity, settlement, foundation selection and design, and developed suitable solutions for real-world geotechnical problems.
PSO2	Life-long learning – To create innovative ideas and solutions for real time problems in structures by adapting the emerging technologies and modern tools.	**Yes**	Adopted emerging technologies such as AI-based prediction, CAD/software tools, numerical analysis and modern geotechnical techniques to develop innovative and sustainable solutions for real-time foundation engineering problems.

OUTCOMES OF PROJECT

 **ESP32-C6-DevKitC-1**	 **Rotary encoder**
 **SPI OLED display**	  **Buzzer, red LED and green LED**

